

SAE 2.01 - 2.02 : Rapport graphes

Première Version

Yann Renard

Tom Cox

Alexandre

8 mai 2026

Table des matières

1	Version 1 : Un seul moyen de transport	3
1.1	Exemple	3
1.2	Modèle pour l'exemple	4
1.3	Modélisation pour la Version 1 dans le cas général	5
1.4	Prototype pour la Version 1	6

1 Version 1 : Un seul moyen de transport

1.1 Exemple

Pour illustrer la Version 1, nous avons choisi un réseau de transport reliant cinq villes du nord de la France : Lille, Dunkerque, Cambrai, Paris et Crépy-en-Valois. Ce réseau comprend dix-huit connexions réparties sur trois modes de transport (train, bus et avion) avec des données de coûts estimées à partir des vitesses moyennes de chaque mode et des barèmes d'émissions carbone de l'ADEME.

Tableau 1 : Données complètes du réseau de transport

Départ	Arrivée	Mode	Prix (€)	CO ₂ (kg)	Durée (min)
Lille	Dunkerque	TRAIN	18	0,8	65
Dunkerque	Lille	TRAIN	18	0,8	65
Lille	Cambrai	TRAIN	14	0,7	55
Cambrai	Lille	TRAIN	14	0,7	55
Cambrai	Paris	TRAIN	35	1,8	95
Paris	Cambrai	TRAIN	35	1,8	95
Paris	Crépy-en-Valois	TRAIN	12	0,5	45
Crépy-en-Valois	Paris	TRAIN	12	0,5	45
Lille	Paris	TRAIN	55	2,4	60
Paris	Lille	TRAIN	55	2,4	60
Dunkerque	Cambrai	TRAIN	22	1,0	80
Cambrai	Dunkerque	TRAIN	22	1,0	80
Dunkerque	Crépy-en-Valois	TRAIN	45	2,0	150
Dunkerque	Cambrai	BUS	12	2,1	110
Cambrai	Crépy-en-Valois	BUS	18	2,8	130
Paris	Crépy-en-Valois	BUS	8	1,2	75
Lille	Paris	AVION	120	85,0	55
Dunkerque	Paris	AVION	110	80,0	50

Table 1.1 : Données complètes du réseau de transport

Les durées intègrent les temps d'attente habituels (enregistrement aéroport pour l'avion, correspondances en gare).

Le scénario retenu pour ce prototype est le suivant :

- Mode de transport : TRAIN uniquement
- Critère d'optimisation : le prix (on cherche à minimiser le coût en euros)
- Ville de départ : Lille
- Ville d'arrivée : Crépy-en-Valois
- Nombre d'itinéraires souhaités : 4

Ce scénario est représentatif d'un voyageur contraint par son budget, souhaitant rejoindre Crépy-en-Valois depuis Lille en train. Le Tableau 2 présente les quatre meilleurs itinéraires identifiés, classés par ordre croissant de prix.

Tableau 2 : Les 4 meilleurs itinéraires TRAIN de Lille à Crépy-en-Valois (critère : prix)

Itinéraire	Prix (€)	Classement
Lille → Cambrai → Paris → Crépy-en-Valois	61	1 ^{er}
Lille → Dunkerque → Crépy-en-Valois	63	2 ^e
Lille → Paris → Crépy-en-Valois	67	3 ^e
Lille → Cambrai → Dunkerque → Crépy-en-Valois	81	4 ^e

Table 1.2 : Les 4 meilleurs itinéraires TRAIN de Lille à Crépy-en-Valois (critère : prix)

Les itinéraires empruntant l'avion ou le bus sont exclus du scénario, le filtre portant exclusivement sur le mode TRAIN.

1.2 Modèle pour l'exemple

Pour résoudre ce problème, on modélise le réseau filtré par un graphe orienté valué. Chaque ville du réseau devient un sommet, et chaque connexion TRAIN devient une arête orientée dont le poids correspond au critère choisi, ici le prix en euros.

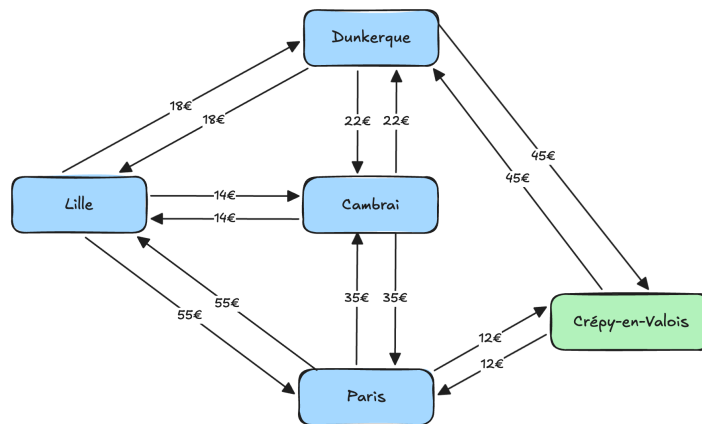


Figure 1.1 : Graphe orienté valué, scénario TRAIN, critère PRIX (Lille vers Crépy-en-Valois)

Dans ce graphe, les cinq sommets sont les villes {Lille, Dunkerque, Cambrai, Paris, Crépy-en-Valois}. Les arêtes orientées et valuées représentent les connexions TRAIN disponibles. À titre d'exemple, l'arête Lille → Paris porte le poids 55 (prix en euros), l'arête Lille → Cambrai porte le poids 14, et l'arête Paris → Crépy-en-Valois porte le poids 12.

Les quatre meilleurs chemins dans ce graphe, exprimés sous forme de suites d'arêtes avec leur poids cumulé, sont :

- **Chemin 1** : Lille → Cambrai → Paris → Crépy-en-Valois, poids total = $14 + 35 + 12 = 61$ €
- **Chemin 2** : Lille → Dunkerque → Crépy-en-Valois, poids total = $18 + 45 = 63$ €
- **Chemin 3** : Lille → Paris → Crépy-en-Valois, poids total = $55 + 12 = 67$ €
- **Chemin 4** : Lille → Cambrai → Dunkerque → Crépy-en-Valois, poids total = $14 + 22 + 45 = 81$ €

L'algorithme KPCC explore les chemins sans cycle dans le graphe et les retourne ordonnés par poids croissant. Les résultats ci-dessus ont été obtenus et vérifiés par l'exécution de la classe `GraphesV1` décrite en Section 1.4.

1.3 Modélisation pour la Version 1 dans le cas général

On se donne un problème de recherche d'itinéraire défini par : un ensemble de villes V , un ensemble de connexions C entre ces villes, une modalité de transport m choisie, un critère d'optimisation c parmi {TEMPS, PRIX, CO2}, une ville de départ s , une ville d'arrivée t , et un entier k représentant le nombre d'itinéraires souhaités. Voici comment ce problème se traduit en termes de graphes.

Type de graphe

On utilise un *multigraphe orienté valué*. L'orientation est nécessaire car une connexion de la ville x vers la ville y n'implique pas l'existence d'une connexion en sens inverse : les lignes de transport sont directionnelles. La valuation permet d'associer à chaque arête le coût

correspondant au critère choisi. Enfin, la structure de multigraphe est justifiée par le fait que deux villes pourraient théoriquement être reliées par plusieurs connexions de même modalité, ce qui correspondrait à des arêtes parallèles dans le graphe.

Sommets du graphe

Les sommets du graphe sont les villes qui apparaissent dans au moins une connexion de la modalité choisie, soit en départ, soit en arrivée. Autrement dit, on ne retient que les villes réellement desservies par le mode de transport sélectionné; les autres sont ignorées. Dans notre exemple, filtrer sur TRAIN exclut toute ville uniquement accessible en bus ou en avion.

Arêtes du graphe et leurs poids

Il existe une arête orientée du sommet x vers le sommet y dans le graphe si et seulement si il existe une connexion de modalité m allant de x à y dans l'ensemble C . Le poids de cette arête est la valeur du critère c pour cette connexion :

- si $c = \text{PRIX}$ → poids = prix en euros
- si $c = \text{TEMPS}$ → poids = durée en minutes
- si $c = \text{CO}_2$ → poids = émissions en kg CO₂e

Cette construction garantit que minimiser le poids total d'un chemin revient exactement à minimiser le critère souhaité par l'utilisateur.

Algorithme de résolution

Une fois le graphe construit, on applique l'algorithme des k plus courts chemins (KPCC) en appelant `AlgorithmeKPCC.kpcc(graphe, s, t, k)`. Cet algorithme retourne une liste ordonnée d'au plus k objets `Chemin`, chacun contenant la liste des arêtes empruntées et le poids total du chemin. Les chemins sont sans cycle et triés par poids croissant, ce qui correspond directement aux k meilleurs itinéraires selon le critère de l'utilisateur.

Cette modélisation est pensée pour permettre une évolution naturelle vers la Version 2 : il suffira d'enrichir le graphe avec des nœuds de correspondance pour prendre en charge les changements de modalité entre deux étapes d'un même voyage.

1.4 Prototype pour la Version 1

La classe `GraphesV1` est une classe Java autonome et exécutable. Conformément aux exigences du sujet, elle ne dépend d'aucune autre classe du projet, à l'exception des interfaces `Lieu` et `Connexion` de la bibliothèque fournie, qu'elle implémente sous forme de classes anonymes directement dans la méthode `main`.

La classe reproduit le scénario décrit en Section 1.1 : elle instancie les cinq villes comme sommets, ajoute les connexions TRAIN avec leurs prix comme poids d'arêtes, puis appelle `AlgorithmeKPCC.kpcc` pour calculer les quatre meilleurs itinéraires de Lille à Crépy-en-Valois. Les résultats sont affichés de façon lisible, avec pour chaque itinéraire le rang, le coût total en euros et le détail des étapes ville par ville.

Lien vers le code source (commit avant la date limite) :

`GraphesV1.java` sur GitLab (équipe E5)